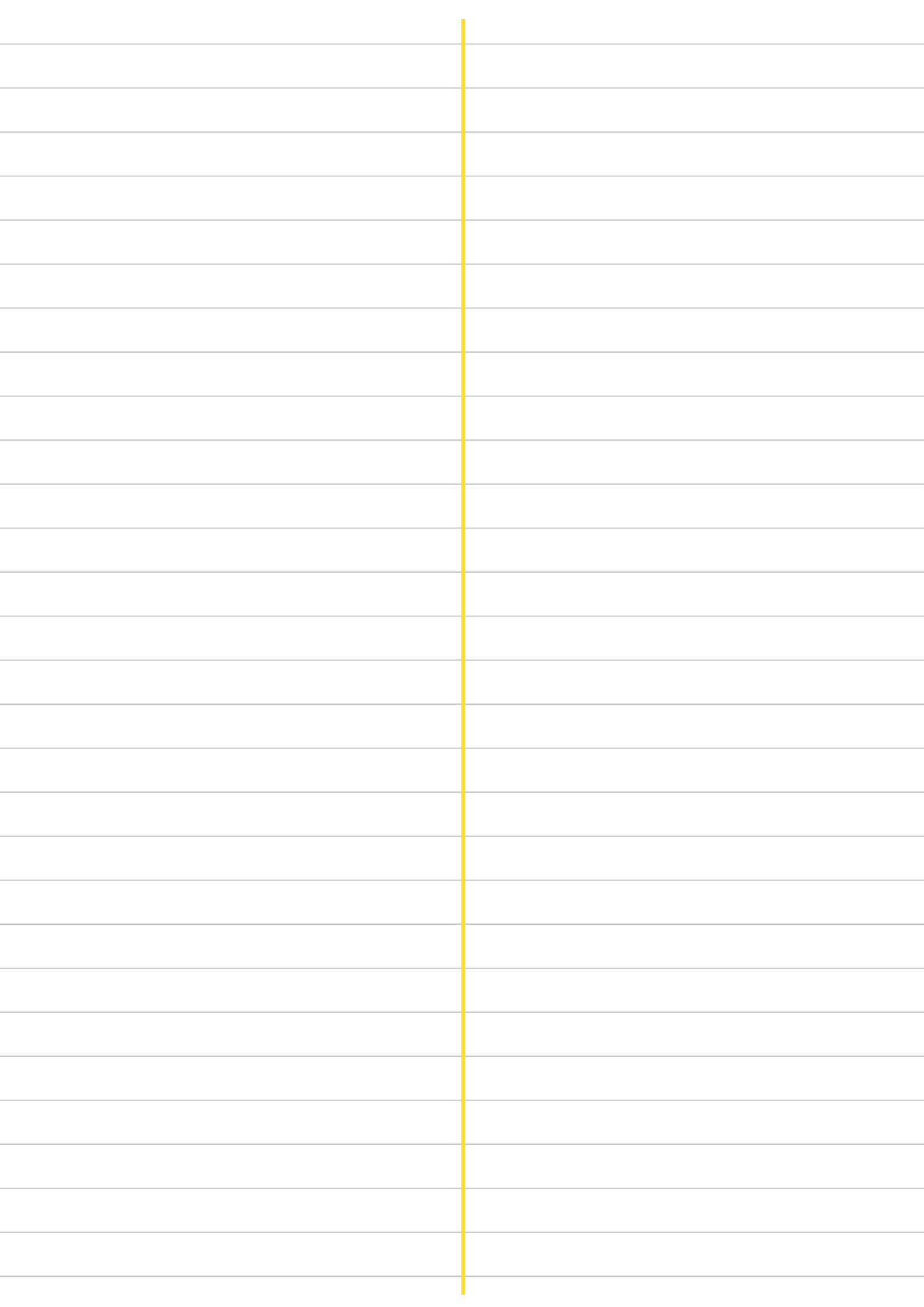




# Unit-3

## General ARM Cortex-M Memory Architecture

### → 32 bit addressing

- ↳  $2^{32}$  unique locations
- ⇒ 4GB [ $1\text{GB} = \text{KB}^3 = 2^{10 \times 3}$ ]
- {from  $0x0000_0000$  to  $0xFFFF\_FFFF$ }

### → Unified mem. space

- ↳ uses Von-Neumann-like arch.
- Where inst. & data share same (code) addr. space.

### → Concurrent Access

- ↳ multiple bus interface although unified.
- ⇒ allows to fetch an inst. and access data simultaneously
- ↳ improves performance.

### → Memory Map of Cortex M4 (next page)

### → Specifics of STM32F407VG

### → Internal mem.

- Flash mem. (1MB): code region
- ↳ non volatile mem. where prog. is stored.

- SRAM (192KB) main volatile mem.
- ↳ 64KB of CCM

- OTP mem. (512 bytes) (one time programmable mem.) for permanent data like serial nos & security keys.

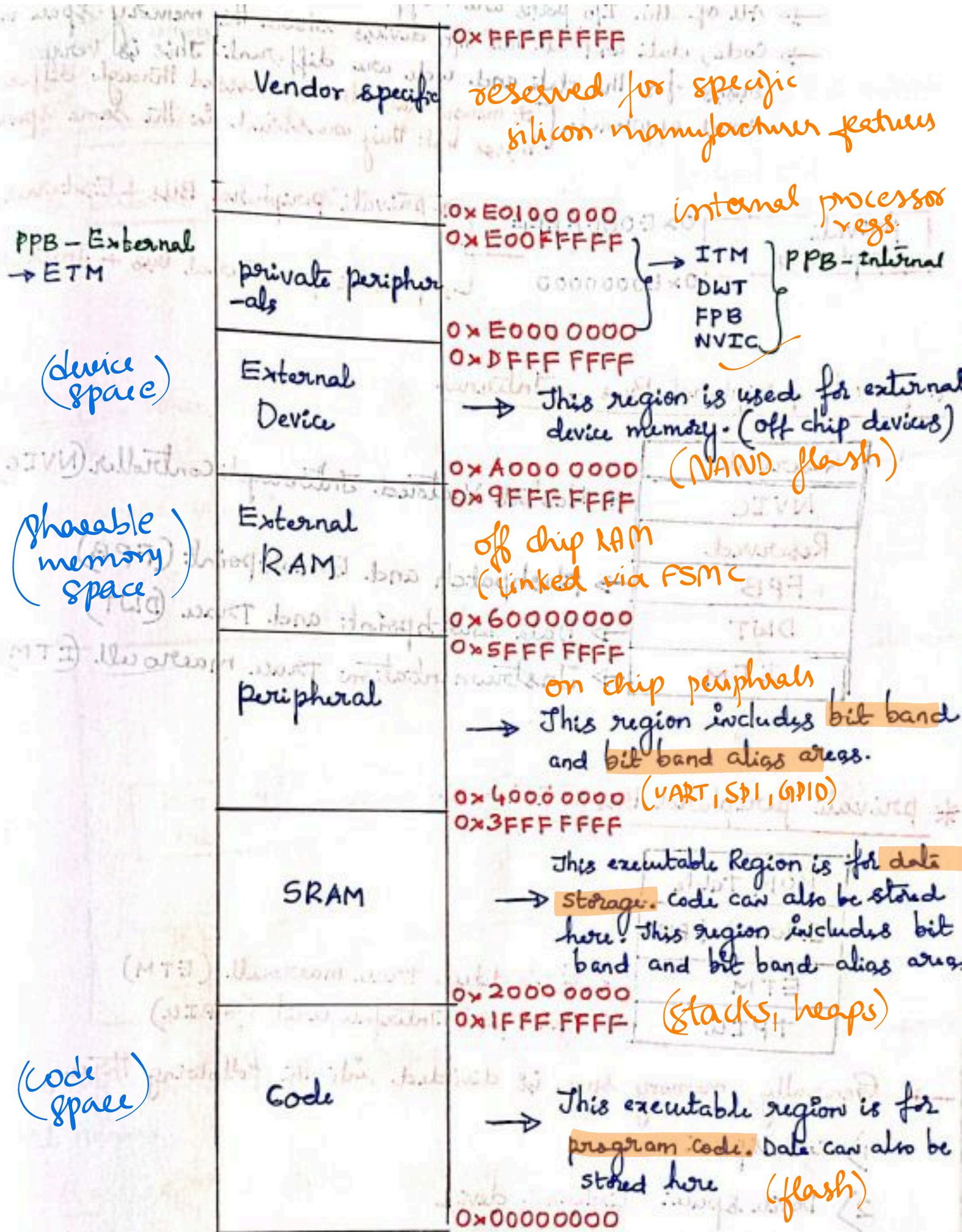
### → CCM (Cortex Coupled Memory)

- ↳ specialised RAM block.
- Zero wait state (faster access)
- Use: ideal for critical tasks like DSP (digital sig. processing)

- FSMC (flexible static mem. controller)
- ↓
- external mem. support through controllers

- SRAM, PSRAM, NOR/NAND flash.

# Memory Mapping



# RCC (Reset and clock controller)

## 1) Reset sys.

- ↳ resets MCU when when faults
- ↳ initialises processor safely occur.

## 2) clk sys.

- ↳ generates clks for CPU & peripherals.
- ↳ controls clk freq. & distribution.

## → Reset Handler

→ first func. executed after reset.

→ Sequence:  $\langle 0x0000_0000 \rightarrow ISP \rangle$

1. Reset occurs  $0x0000_0004$
2. CPU fetches reset vector
3. CPU jumps to Reset\_Handler
4. Stack initialised
5. Data sections initialised
6. main() func. called.

## → Types of Reset Sources

- Power-on reset
- Manual reset (NRST)
- Independent Watchdog reset
- Brown out reset

## → NRST (negative reset)

- ↳ active low reset if pin

NRST value

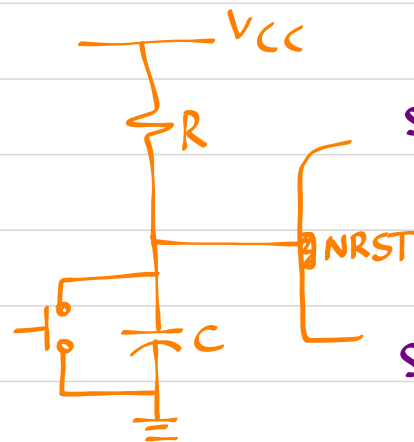
- HIGH → MCU runs normally
- LOW → MCU resets

## → POR (Power on reset)

- When power is first applied
  - ↳ voltage rises from 0V
  - ↳ internal reset circuit holds MCU in reset.
  - ↳ MCU starts only after voltage becomes stable

## → Manual reset

- board has push button connected to NRST
- when pressed,
  - NRST = LOW → MCU resets when released
  - NRST = HIGH → MCU restarts



power ON:

- S1) C → short circuit  
⇒ NRST = LOW  
Reset active
- S2) C → charges slowly through R.  
⇒ V ↑

Why use RC?

- delay starting slightly
- Ensure stable power before execution.
- S3) V crosses threshold  
NRST = HIGH  
MCU starts execution



→ Independent Watchdog reset (IWDG)

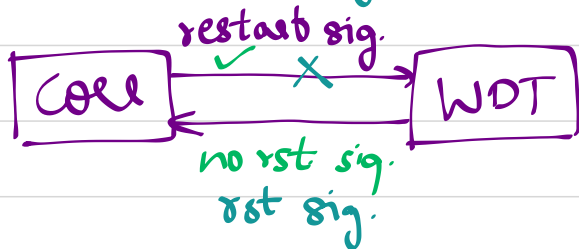
→ Purpose:

→ prog. crashes (unstable sys)  
↓  
infinite loop occurs → CPU hangs

→ Watchdog prevents this  
↓  
timer continuously counts down.

→ if software healthy,  
CPU periodically refreshes/  
restarts watchdog timer.  
(WDT)

→ if CPU hangs:  
No refresh occurs  
timer reaches 0  
Watchdog generates reset.



PH → ensures min. rst pulse width (~20µs)  
↓  
as rst pulse must last long enough for all hardware blocks to rst properly.

→ Brown out reset (BOR)

when supply voltage drops below safe operating level  
{eg: weak battery, sudden power fluctuation}

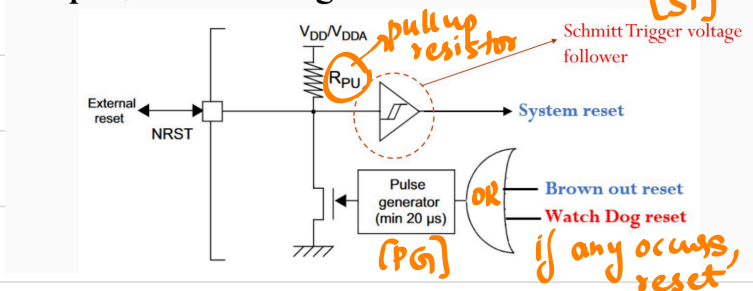
→ BOR working.  
↓

MCU continuously monitors VDD.

→ if  $VDD < \text{threshold}$   
MCU forced into reset state

→ when voltage becomes normal,  
MCU restarts safely

Simplified block diagram of Reset Circuits (ST)



Rpu → keeps NRST HIGH normally  
↳ w/o Rpu, pin may float  
false rst possible.

ST → cleans noisy sig.  
→ converts unstable analog sigs.  
to sharp digital sigs.  
imp as rst sigs must be clean and reliable.

## → Clocks

→ digital sys. operate synch.

→ clk controls:

- cpu speed,
- peripheral timings
- comm. baud rates
- timers
- ADC sampling

no clk = no operation

→ Types of clk sources

- 1) HSI (High Speed Internal)  
16 MHz Internal RC oscillator
- 2) HSE (High Speed External)  
4-26 MHz External LC oscillator
- 3) PLL (Phase locked loop)  
freq. multiplier/divider

→ Secondary clk sources.

→ LSI (low speed internal)  
32 kHz internal RC osc

→ LSE (low speed external)  
32.768 kHz external crystal

$$2^{15} = 32768$$

easy to divide into exact  
1 second timing.

## (NOTE)

stm32 cubemx writes configurations  
into regs.

→ Embedded programming is  
fundamentally,  
configuring hardware regs.

→ GPIO (General Purpose  
Input Output)

→ interface b/w microcontroller  
& external world

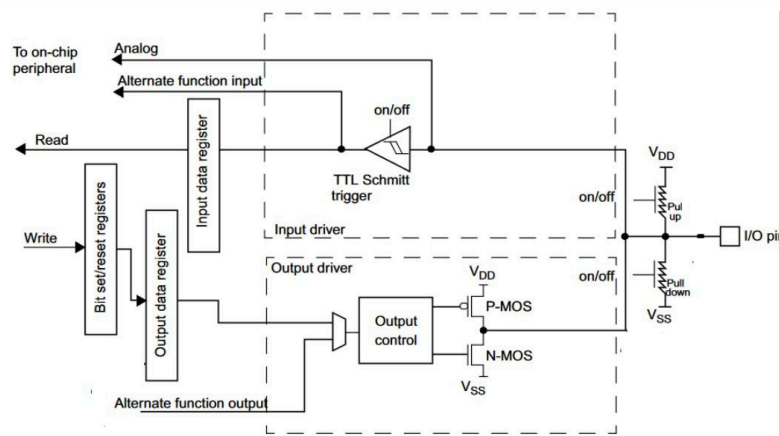
→ Most stm32 pins work either  
as GPIO or alternate func.

→ Naming: PA0  
port A pin 0

→ 16 pins per port

port A → PA0 to PA15

in 100-pin package, → port A to E  
146-pin LQFP → port A to I



## GPIO Pin internal structure

internally contains

- 1) i/p driver
- 2) o/p driver
- 3) pullup. resistor
- 4) pull down res.
- 5) schmitt trigger
- 6) resistors

→ i/p driver → when pin  
configured as i/p

- reads ext. voltage level
- sends value to internal logic

→ o/p driver → when pin  
configured as o/p

- drives HIGH / LOW voltage into pin
- implemented using PMOS / NMOS transistors.

→ Schmitt trigger [ST]

- cleans noisy i/p sigs.
- prevents unstable switching

→ Pullup / pull down res.  
- weak internal res.

pullup → connects pin weakly to VDD

Default state — HIGH

pull down → connects pin weakly to GND

Default state — LOW

→ why needed?

- say switch is open,
- w/o pull res., pin floats, random values occur

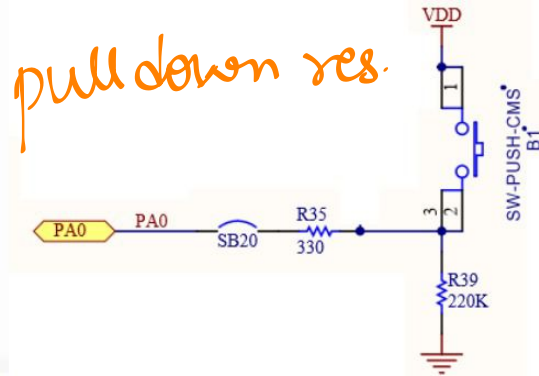
[floating i/p]  
very dangerous to digital sys

→ i/p operation:

- o/p buffer disabled.
- pin does NOT drive voltage only reads sigs.
- ST enabled (i/p passes through ST before entering MCU)
- i/p value stored in GPIOx\_IDR

(i/p data reg.)

- GPIO\_MODE\_INPUT
- GPIO\_NOPULL → no internal pull res.



→ button NOT pressed →  
PA0 connected to GND through resistor.  $\Rightarrow PA0 = 0$

→ button pressed →  
PA0 connected to VDD  
 $\Rightarrow PA0 = 1$

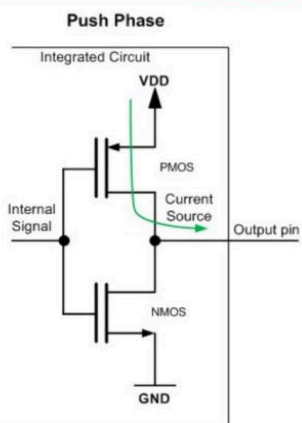
→ GPIO\_NOPULL

↳ as external pull down res. already exists.

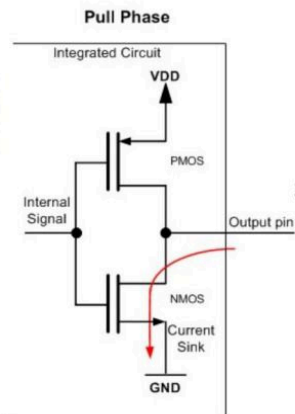
→ o/p operation.

- o/p buffer enabled
- MCU can drive pin HIGH or LOW
- 2 major o/p types
  - ↳ push-pull
  - ↳ open drain

Push Pull:



Pin state=1



Pin state=0

→ push-pull

- uses both PMOS & NMOS

PMOS ON

PMOS OFF

NMOS OFF

NMOS ON

$V_{out} = V_{DD}$

$V_{out} = 0$  (GND)

Pin state = 1

Pin state = 0

→ adv: fast switching  
strong HIGH / LOW  
no ext. res. needed

→ use: LEDs, SPI, clk sigs., digital o/p.

→ open drain o/p.

- uses only NMOS

NMOS ON

NMOS OFF

$V_{out} = 0$

$V_{out} = \text{Hi-Z}$

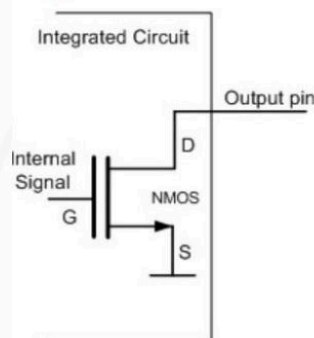
(high impedance)  
neither HIGH nor LOW

→ pull up (ext. or internal)

↳ needed to create HIGH level.

→ use: I2C

Open drain:



Internal or external pull up is required when pin state is Hi-Z

When,

NMOS is on, pin state=0

NMOS is off, pin is floating (Hi-Z state)

\*Hi-Z is high impedance state



• **GPIO port mode register (GPIOx\_MODER)** *controls pin mode*

|              |    |              |    |              |    |              |    |              |    |              |    |             |    |             |    |
|--------------|----|--------------|----|--------------|----|--------------|----|--------------|----|--------------|----|-------------|----|-------------|----|
| 31           | 30 | 29           | 28 | 27           | 26 | 25           | 24 | 23           | 22 | 21           | 20 | 19          | 18 | 17          | 16 |
| MODER15[1:0] |    | MODER14[1:0] |    | MODER13[1:0] |    | MODER12[1:0] |    | MODER11[1:0] |    | MODER10[1:0] |    | MODER9[1:0] |    | MODER8[1:0] |    |
| rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw          | rw | rw          | rw |
| 15           | 14 | 13           | 12 | 11           | 10 | 9            | 8  | 7            | 6  | 5            | 4  | 3           | 2  | 1           | 0  |
| MODER7[1:0]  |    | MODER6[1:0]  |    | MODER5[1:0]  |    | MODER4[1:0]  |    | MODER3[1:0]  |    | MODER2[1:0]  |    | MODER1[1:0] |    | MODER0[1:0] |    |
| rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw           | rw | rw          | rw | rw          | rw |

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)  
 These bits are written by software to configure the I/O direction mode.  
 00: Input (reset state)  
 01: General purpose output mode  
 10: Alternate function mode  
 11: Analog mode

*UART, SPI, I2C, TIMERS* ← *4 modes*

• **GPIO port input data register (GPIOx\_IDR)** → *contains actual logic level on pins*

|          |       |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
|----------|-------|-------|-------|-------|-------|------|------|------|------|------|------|------|------|------|------|
| 31       | 30    | 29    | 28    | 27    | 26    | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Reserved |       |       |       |       |       |      |      |      |      |      |      |      |      |      |      |
| 15       | 14    | 13    | 12    | 11    | 10    | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| IDR15    | IDR14 | IDR13 | IDR12 | IDR11 | IDR10 | IDR9 | IDR8 | IDR7 | IDR6 | IDR5 | IDR4 | IDR3 | IDR2 | IDR1 | IDR0 |
| r        | r     | r     | r     | r     | r     | r    | r    | r    | r    | r    | r    | r    | r    | r    | r    |

Bits 31:16 Reserved, must be kept at reset value. *if (GPIOA→IDR & (1<<0)) → checks whether PA0 is HIGH*

Bits 15:0 **IDRy**: Port input data (y = 0..15)  
 These bits are read-only and can be accessed in word mode only. They contain the input value of the corresponding I/O port.

**GPIO port bit set/reset register (GPIOx\_BSRR)** → *used to set/reset pin automatically* !!

|      |      |      |      |      |      |     |     |     |     |     |     |     |     |     |     |
|------|------|------|------|------|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 31   | 30   | 29   | 28   | 27   | 26   | 25  | 24  | 23  | 22  | 21  | 20  | 19  | 18  | 17  | 16  |
| BR15 | BR14 | BR13 | BR12 | BR11 | BR10 | BR9 | BR8 | BR7 | BR6 | BR5 | BR4 | BR3 | BR2 | BR1 | BR0 |
| w    | w    | w    | w    | w    | w    | w   | w   | w   | w   | w   | w   | w   | w   | w   | w   |
| 15   | 14   | 13   | 12   | 11   | 10   | 9   | 8   | 7   | 6   | 5   | 4   | 3   | 2   | 1   | 0   |
| BS15 | BS14 | BS13 | BS12 | BS11 | BS10 | BS9 | BS8 | BS7 | BS6 | BS5 | BS4 | BS3 | BS2 | BS1 | BS0 |
| w    | w    | w    | w    | w    | w    | w   | w   | w   | w   | w   | w   | w   | w   | w   | w   |

Bits 31:16 **BRy**: Port x reset bit y (y = 0..15)  
 These bits are write-only and can be accessed in word, half-word or byte mode. A read to these bits returns the value 0x0000.  
 0: No action on the corresponding ODRx bit  
 1: Resets the corresponding ODRx bit  
*Note: If both BSx and BRx are set, BSx has priority.*

Bits 15:0 **BSy**: Port x set bit y (y = 0..15)  
 These bits are write-only and can be accessed in word, half-word or byte mode. A read to these bits returns the value 0x0000.  
 0: No action on the corresponding ODRx bit  
 1: Sets the corresponding ODRx bit

*GPIO0→BSRR = (1<<15); sets PD15 HIGH*  
*= (1<<(15+16)); sets PD15 LOW*

**RCC AHB1 peripheral clock enable register (RCC\_AHB1ENR)**

|          |             |         |             |            |            |          |          |         |         |         |         |              |         |           |          |
|----------|-------------|---------|-------------|------------|------------|----------|----------|---------|---------|---------|---------|--------------|---------|-----------|----------|
| 31       | 30          | 29      | 28          | 27         | 26         | 25       | 24       | 23      | 22      | 21      | 20      | 19           | 18      | 17        | 16       |
| Reserved | OTGHSULPIEN | OTGHSEN | ETHMACPTPEN | ETHMACRXEN | ETHMACTXEN | ETHMACEN | Reserved |         |         | DMA2EN  | DMA1EN  | CCMDATARAMEN | Res.    | BKPSRAMEN | Reserved |
|          | rw          | rw      | rw          | rw         | rw         | rw       |          |         |         | rw      | rw      |              |         | rw        |          |
| 15       | 14          | 13      | 12          | 11         | 10         | 9        | 8        | 7       | 6       | 5       | 4       | 3            | 2       | 1         | 0        |
| Reserved |             |         | CRCEEN      | Reserved   |            |          | GPIOIEN  | GPIOHEN | GPIOGEN | GPIOFEN | GPIOEEN | GPIODEN      | GPIOCEN | GPIOBEN   | GPIOAEN  |
|          |             |         | rw          |            |            |          | rw       | rw      | rw      | rw      | rw      | rw           | rw      | rw        | rw       |

→ *GPIOA ⇒ bit 0*

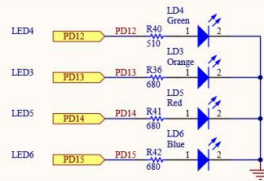
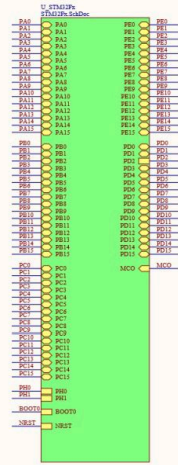
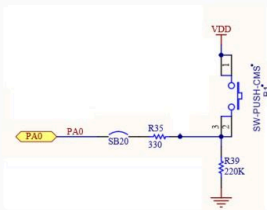
Bit 0 **GPIOAEN**: IO port A clock enable  
 Set and cleared by software.  
 0: IO port A clock disabled  
 1: IO port A clock enabled

*Before using GPIO, clk must be enabled*  
*RCC→AHB1ENR |= (1<<0); bit 0 = GPIOA enable*

## Port Programming

- Design STM32F407VG based to connect push button and LED. Write program to read Push button status and toggle LED.

### Interfacing diagram:



30

MGR,ECE,RVCE

### Program(register programming)

```
#include "stm32f4xx.h"
void delayMs(int n);
void main() {
    RCC->AHB1ENR |= 9;
    GPIOD->MODER |= 0x40000000;
    while(1) {
        if((GPIOA->IDR & 0x0000 0001) == 1) {
            GPIOD->BSRR = 1 << 15;
        } else {
            GPIOD->BSRR = 1 << 31;
        }
    }
}
```

*Handwritten notes:*

- `#include "stm32f4xx.h"` → **Header file with register declarations**
- `RCC->AHB1ENR |= 9;` → **enable Clock to GPIOA & GPIOD** (bit 0, bit 3)
- `GPIOD->MODER |= 0x40000000;` → **Set PD15 as GPIO output & PA0 act as input by default** (bit 0, bit 3)
- `if((GPIOA->IDR & 0x0000 0001) == 1)` → **PA0** (bit 0)
- `GPIOD->BSRR = 1 << 15;` → **PD15 set** (bit 15)
- `GPIOD->BSRR = 1 << 31;` → **PD15 reset** (bit 31)

$RCC \rightarrow AHB1ENR |= (1 \ll 0) | (1 \ll 3);$

$GPIOD \rightarrow MODER |= (1 \ll 30);$

$GPIOD \rightarrow MODER \&= \sim(1 \ll 31);$

### Program(using STM32CubeMx):

- Assume that, STM32CubeMx is used and PA0 is configured as GPIO\_Input(No Pullup/Pull down) and PD15 as GPIO\_Output(Output push pull and No Pullup/Pull down) and HAL is generated.

### Application code:

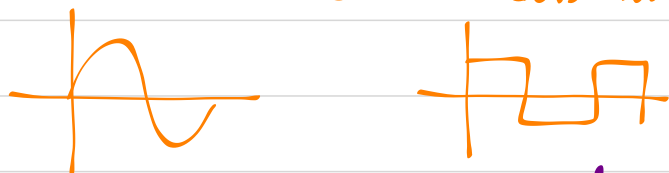
```
#include "main.h"
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    while (1)
    {
        if(HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0)==GPIO_PIN_SET)
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15,GPIO_PIN_SET);
        else
            HAL_GPIO_WritePin(GPIOD,GPIO_PIN_15,GPIO_PIN_SET);
    }
}
```

*Handwritten notes:*

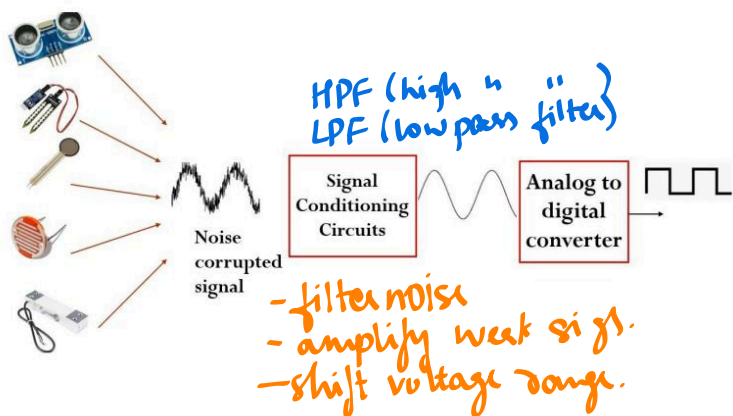
- `HAL_Init();` → **library**
- `SystemClock_Config();` → **clk**
- `MX_GPIO_Init();` → **moder**
- `HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0)==GPIO_PIN_SET` → **if**
- `HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15,GPIO_PIN_SET);` → **write**
- `HAL_GPIO_WritePin(GPIOD,GPIO_PIN_15,GPIO_PIN_SET);` → **write**

1

can understand.



- eg: 23V  $\rightarrow$  01001010



- how small a voltage change the ADC can detect.

- $$\text{Step size} = \frac{V_{IFS}}{2^n} \rightarrow \text{full scale voltage}$$

$\uparrow$  resolution  $\leftrightarrow$   $\downarrow$  step size

eg: 5V full scale; ADC 8bit

Resolution =  $\frac{5}{256} = 0.01953V$  ;  $\Rightarrow$  ADC can detect 19.5mV per step.

- ADC rounds analog value to nearest digital level

- ↓
- Quant. error.

$$\text{max error} = \pm \frac{1}{2} \text{LSB}$$

$\Rightarrow$  actual voltage may differ by half a stepsize.

$d = \frac{e}{\text{Resolution}}$   $\rightarrow$  analog yp

digital o/p code

→ eg: 4 bit ADC; full scale = 8V  
voltage

$$\text{step size} = \frac{8}{2^4} = 0.5V$$

- |      |   |      |
|------|---|------|
| - 0V | - | 0000 |
| 0.5V | - | 0001 |
| 1V   | - | 0010 |
| 1.5V | - | 0011 |
| ⋮    |   |      |
| 7.5V | - | 1111 |



## → Sampling:

- ADC samples the sig. at discrete times  
 $t_1, t_2, t_3$
- Each sample converted to binary  
eg. at  $t_1 \rightarrow$  o/p may be 0100.

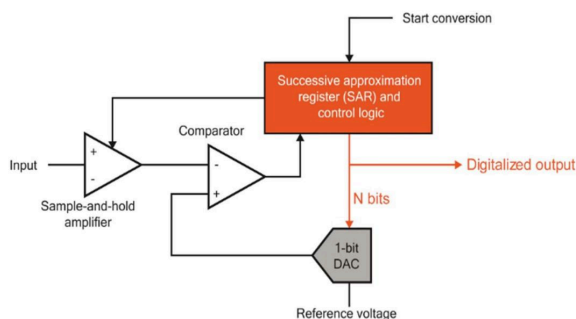
## → Aliasing:

- If sampling rate is too low, ADC creates false freq. data called aliases
- aliasing.
- use anti-aliasing LPF bc ADC do prevent it.

## → Types of ADC:

- SAR
- Delta-sigma
- Dual-slope
- flash
- pipeline

## 1) SAR ADC (Successive approximation ADC)



## Process:

1. Input sampled
2. SAR guesses digital code
3. DAC converts guess to analog
4. Comparator compares:
  - input voltage
  - DAC output
5. SAR adjusts bits
6. Repeats until final digital value obtained

- Range: 0-15V; 8 bit DAC  
ip = 5V

Resolution:  $15/256 = 0.058853$ ; Digital Encoding =  $5/0.058853 = 85d = 55h = 01010101$

|                                          |                                                                           |                                                                                                          |   |   |   |   |   |   |   |   |                                                                     |                                                                                                          |   |   |   |   |   |   |   |   |
|------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|
| $\frac{1510}{2} = 755 > 5 \rightarrow 0$ | $\frac{1}{2}(V_{\max} + V_{\min}) = 7.5$ volts<br>$V_{\min} = 7.5$ volts. | <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table> | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | $\frac{1}{2}(5.63 + 4.69) = 5.16$ volts<br>$V_{\min} = 5.16$ volts. | <table><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table> | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0                                        | 0                                                                         | 0                                                                                                        | 0 | 0 | 0 | 0 | 0 |   |   |   |                                                                     |                                                                                                          |   |   |   |   |   |   |   |   |
| 0                                        | 1                                                                         | 0                                                                                                        | 1 | 0 | 0 | 0 | 0 |   |   |   |                                                                     |                                                                                                          |   |   |   |   |   |   |   |   |
| $\frac{750}{2} = 375 < 5 \rightarrow 1$  | $\frac{1}{2}(7.5 + 0) = 3.75$ volts<br>$V_{\min} = 3.75$ volts.           | <table><tr><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table> | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | $\frac{1}{2}(5.16 + 4.69) = 4.93$ volts<br>$V_{\min} = 4.93$ volts. | <table><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr></table> | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0                                        | 1                                                                         | 0                                                                                                        | 0 | 0 | 0 | 0 | 0 |   |   |   |                                                                     |                                                                                                          |   |   |   |   |   |   |   |   |
| 0                                        | 1                                                                         | 0                                                                                                        | 1 | 0 | 1 | 0 | 0 |   |   |   |                                                                     |                                                                                                          |   |   |   |   |   |   |   |   |
| $\frac{375}{2} = 187 > 5 \rightarrow 0$  | $\frac{1}{2}(7.5 + 3.75) = 5.63$ volts<br>$V_{\min} = 5.63$ volts.        | <table><tr><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table> | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | $\frac{1}{2}(5.16 + 4.93) = 5.05$ volts<br>$V_{\min} = 5.05$ volts. | <table><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr></table> | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0                                        | 1                                                                         | 0                                                                                                        | 0 | 0 | 0 | 0 | 0 |   |   |   |                                                                     |                                                                                                          |   |   |   |   |   |   |   |   |
| 0                                        | 1                                                                         | 0                                                                                                        | 1 | 0 | 1 | 0 | 0 |   |   |   |                                                                     |                                                                                                          |   |   |   |   |   |   |   |   |
| $\frac{187}{2} = 93 < 5 \rightarrow 0$   | $\frac{1}{2}(5.63 + 3.75) = 4.69$ volts<br>$V_{\min} = 4.69$ volts.       | <table><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table> | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | $\frac{1}{2}(5.05 + 4.93) = 4.99$ volts                             | <table><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr></table> | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| 0                                        | 1                                                                         | 0                                                                                                        | 1 | 0 | 0 | 0 | 0 |   |   |   |                                                                     |                                                                                                          |   |   |   |   |   |   |   |   |
| 0                                        | 1                                                                         | 0                                                                                                        | 1 | 0 | 1 | 0 | 1 |   |   |   |                                                                     |                                                                                                          |   |   |   |   |   |   |   |   |

- half search range

### Pro's

- Simple circuit with only one comparator needed
- Higher sample rates are possible compared to delta-sigma ADCs
- Handles natural and unnatural waveshapes well

### Cons

- Anti-aliasing filtering must be added externally
- Bit resolution and dynamic range limited compared to delta-sigma ADCs

### Applications

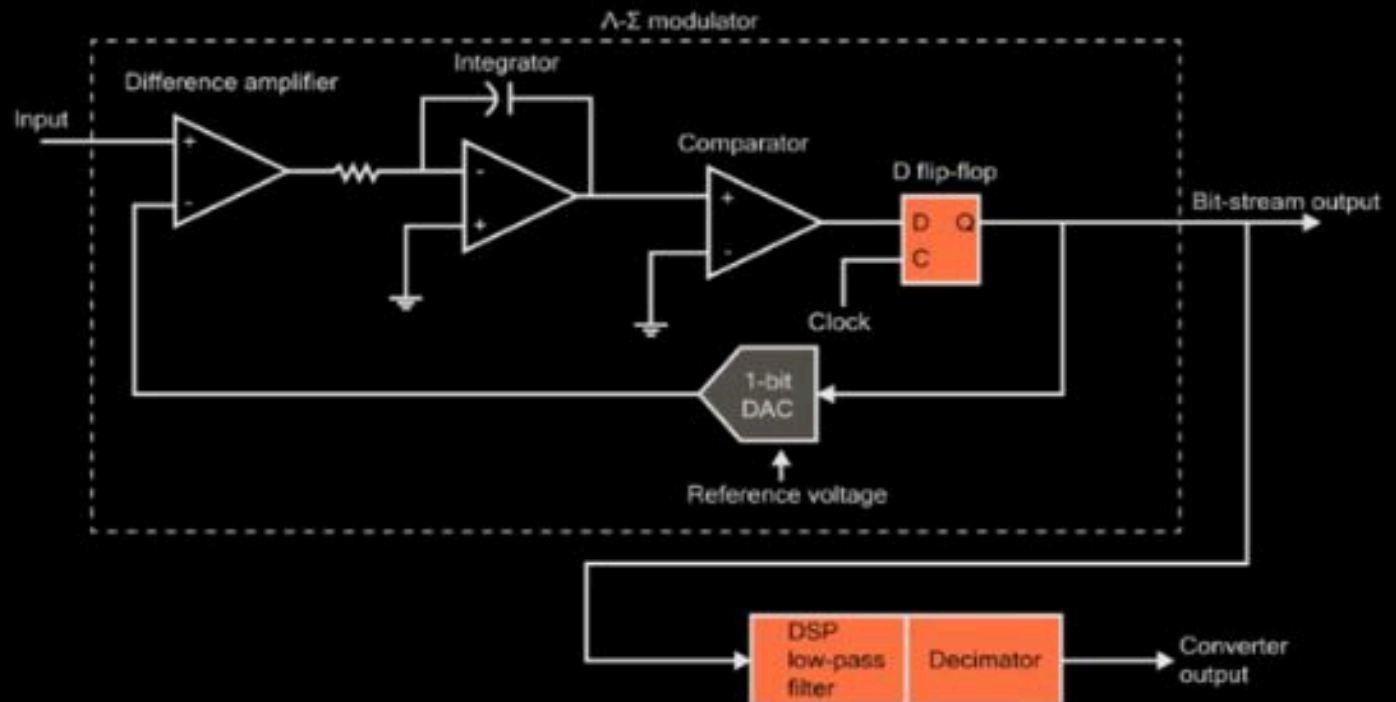
Applications for SAR ADCs include DAQ systems, from low-end multiplexed ADC systems to higher speed single ADC per channel systems, industrial control and measurement, and CMOS imaging

## 2) Delta-sigma ( $\Delta\Sigma$ ) ADC

im too lazy to write ....

will attach clande notes yayy.





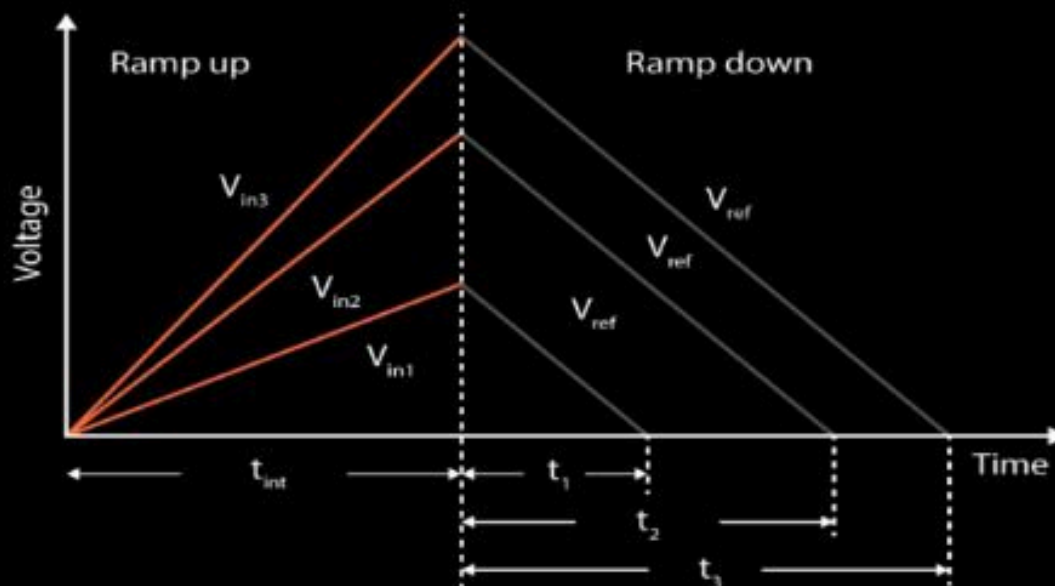
```

graph TD
    A[Input voltage] --> B[Difference amplifier computes error]
    B --> C[Integrator accumulates error]
    C --> D[Comparator creates 1-bit output]
    D --> E[Flip-flop synchronizes bits]
    E --> F[1-bit DAC creates feedback voltage]
    F --> G[Loop continuously self-corrects]
    G --> H[Bit density represents analog value]
    H --> I[Digital filter averages stream]
    I --> J[Decimator lowers data rate]
    J --> K[Final digital output]

```

## Dual Slope ADC:

ADC...



## BIG IDEA

A Dual Slope ADC works in TWO phases:

| Phase     | Action                                  |
|-----------|-----------------------------------------|
| Ramp Up   | Integrate unknown input voltage         |
| Ramp Down | Use reference voltage to return to zero |

Then:

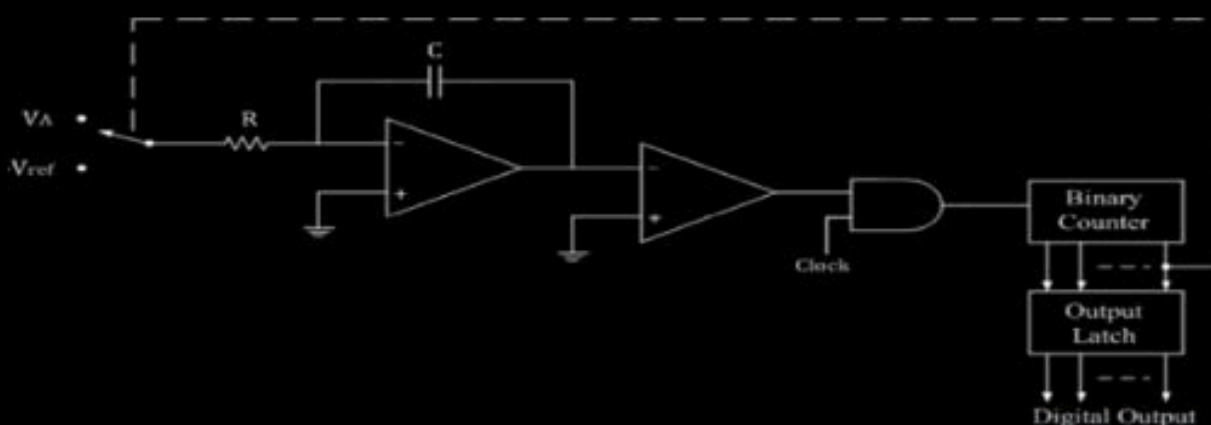
Measured discharge time  $\propto$  Input voltage



Integration time is fixed for all measurements

## Dual Slope ADC:

ADC...



## Step 1

Apply unknown input voltage.

## Step 2

Integrator ramps upward for fixed time.

## Step 3

Switch changes to negative reference voltage.

## Step 4

Integrator ramps downward.

## Step 5

Counter counts clock pulses during ramp-down.

## Step 6

Comparator detects zero crossing.

## Step 7

Counter stops.

Count value becomes digital output.

Flash ADC:

- Fastest type of ADC
- Converts analog to digital voltage in one step
- Flash ADC compares input voltage to many reference voltages simultaneously

### Why Flash ADC is extremely fast

Other ADCs:

| ADC Type    | Method                 |
|-------------|------------------------|
| SAR         | One comparison per bit |
| Dual slope  | Time measurement       |
| Delta-sigma | Oversampling           |

But Flash ADC:

All comparisons happen simultaneously



So conversion time is extremely small.



## VERY IMPORTANT RULE

A Flash ADC requires:

$$2^N - 1$$

comparators.

**Here:**

For 2-bit ADC:

$$2^2 - 1 = 3$$

comparators.

And indeed diagram shows:

c1, c2, c3



- 
- Comparator outputs form smth called the thermometer code:
- Suppose  $v_{in} < v/4$  then c1,c2 and c3 output 000-thermometer code
- Binary output will be 00
- Therefore to convert the thermometer code to binary output we need a encoder logic-logic is called the priority encoder

### Complete Conversion Table

| Vin Range    | C3 C2 C1 | B1 B0 |
|--------------|----------|-------|
| < V/4        | 000      | 00    |
| V/4 to 2V/4  | 001      | 01    |
| 2V/4 to 3V/4 | 011      | 10    |
| > 3V/4       | 111      | 11    |





Analog input applied

↓

Resistor ladder generates thresholds

↓

All comparators compare simultaneously

↓

Comparator outputs create thermometer code

↓

Encoder converts to binary

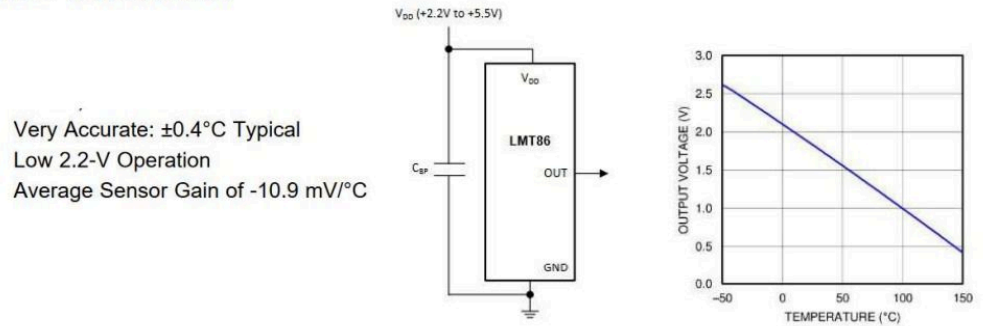
↓

Digital output obtained

| ADC TYPE                       | PROS                                                        | CONS                                 | MAX RESOLUTION | MAX SAMPLE RATE | MAIN APPLICATIONS                          |
|--------------------------------|-------------------------------------------------------------|--------------------------------------|----------------|-----------------|--------------------------------------------|
| Successive Approximation (SAR) | Good speed/resolution ratio                                 | No inherent anti-aliasing protection | 18 bits        | 10 MHz          | Data Acquisition                           |
| Delta-sigma ( $\Delta\Sigma$ ) | High dynamic performance, inherent anti-aliasing protection | Hysteresis on unnatural signals      | 32 bits        | 1 MHz           | Data Acquisition, Noise & Vibration, Audio |
| Dual Slope                     | Accurate, inexpensive                                       | Low speed                            | 20 bits        | 100 Hz          | Voltmeters                                 |
| Pipelined                      | Very fast                                                   | Limited resolution                   | 16 bits        | 1 GHz           | Oscilloscopes                              |
| Flash                          | Fastest                                                     | Low bit resolution                   | 12 bits        | 10 GHz          | Oscilloscopes                              |

## Questions

- STM32F407VG MCU is hosting two 12 bits ADCs with analog input range 0 V to 3.3 V.
- 1) Calculate the resolution(step size) and quantization error.
- If ADC output is 0x100 with ADC specs as shown previous question, calculate the analog output voltage.
- 2) Identify different ways to increase resolution of ADC.
- 3) The figure below shows LMT86 Temperature sensor, corresponding IO characteristics and minimal specs. Comment on ADC selection.
- 4)



1)  $\rightarrow \text{step size} = \frac{3.3}{2^{12}}$

Resolution =  $0.805 \text{ mV}$

$\rightarrow \text{quan. error} = \pm \frac{1}{2} \text{ LSB}$

$= \pm \frac{1}{2} (0.805)$

$q_e = \pm 0.402 \text{ mV}$

2)  $\text{O/p} = 0 \times 100$   
 $\text{O/p} = 256$

$\rightarrow V_{in} = \text{digital code} \times \text{Resolution}$   
 $= 256 \times 0.805 \text{ mV}$   
 $V_{in} = 0.206 \text{ V}$

- 3) To  $\uparrow$  resolution (making more precise)
- $\uparrow$  no. of bits
  - $\downarrow$  i/p voltage  $(\downarrow) = \frac{V(\downarrow)}{2^{n(\uparrow)}}$   
 $(0-1V)$  gives better precision than  $0-5V$
  - use oversampling  
 take many samples & average them.  
 $\downarrow$   
 improves effective resolution
  - reduce noise
  - sig. amplification
  - precision ref. voltage

4)

→ STM32F407V67

→ 12bit SAR ADC

→ 19 multiplexed channels

→ 16 ext. analog i/p.s.

↳ ADC-INO to ADC-IN15  
↳ connected to GPIO pins

→ power supply

↳ VDDA (connected to 3.3V)  
↳ VSSA

→ reference voltage

↳ compares i/p sig. w/ ref. voltage.

$-V_{ref} \leq V_{in} \leq +V_{ref}$

(0V) (3.3V) usually

→ ADC DR (data reg)

↳ digital value stored in DR

↳ for 12 bit ADC,

value ranges from 0 to 4095.

→ EOC (End of conversion)

= 1 when conversion finishes

⇒ data ready in DR.

→ ADC trigger methods

→ SWSTART bit in ADCSR2

↳ software sets, conv. begins

→ TIM1-CH1, TIM3-TRGO

↳ timer events trigger ADC automatically

→ ADCCLK

APB2 clk2

→ derived from PCLK2  
using prescaler.

→ options: — PCLK2/2, /4, /6, /8  
default

→ conversion time formula

$$T_{conv} = T_{sampling} + 12 \text{ cycles}$$

eg: — ADCCLK = 30 MHz

sampling time = 3 cycles.

$$T_{conv} = 3 + 12 = 15 \text{ cycles}$$

$$\text{clk period} = \frac{1}{30 \text{ MHz}} = 3.33 \text{ ns}$$

$$\text{Tot. time} = 15 \times 3.33 \text{ ns}$$

$$= 0.5 \mu\text{s}$$

→ sampling time options: —

3, 15, 28, 56, 84, 112, 144,

480 cycles

DAC (Digital to Analog Converter) — takes binary no. and generates equi. analog voltage

#### R-2R DAC:

- Main idea is using two resistors  $r$  and  $2r$  you get your analog voltage from the digital input

#### BIG IDEA

Each binary bit contributes a weighted amount of current/voltage.

#### Important Binary Concept

In binary:

| Bit Position | Weight      |
|--------------|-------------|
| MSB          | Largest     |
| Next bit     | Half of MSB |
| Next bit     | Half again  |
| LSB          | Smallest    |

- Using  $r$ - $2r$  continuously creates binary current division

#### Understanding the Ladder

The resistor network repeatedly splits current by 2.

So each stage contributes:

| Bit  | Weight |
|------|--------|
| MSB  | $1/2$  |
| Next | $1/4$  |
| Next | $1/8$  |
| LSB  | $1/16$ |

- The op amp acts as a current to voltage converter
- The circuit produces weighted current so the op amp converts the total current into output voltage

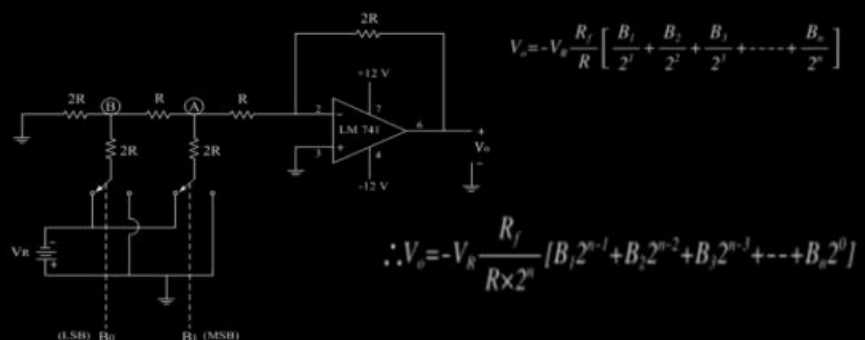
#### Why MSB contributes most

MSB current encounters fewer divisions.

So more current reaches op-amp.

LSB undergoes more splitting.

So smaller contribution.



#### Example — 2-bit DAC

Suppose:

$V_{ref} = 4V$   
 $R_f = R$

Input:

$B_1 B_0 = 10$

#### Formula

$$V_o = -4 \left( \frac{1}{2} + \frac{0}{4} \right)$$

#### Result

$$V_o = -2V$$



